FULL NAME:	[Enter full name]
DATE:	[Enter submission date]
REGISTRATION NUMBER:	[Enter registration number]
COURSE:	COS 202 (Programming II)

1.0 SMART CLINIC APPLICATION PROJECT REPORT

1.1 PROJECT DESCRIPTION AND FEATURES

The Smart Clinic Appointment and Patient Management System (SCAPMS 3.0) is a C++17 desktop application for the COS 202 lab assessment. Its Qt Widgets interface provides patient registration, appointment management, a FIFO queue, searching, sorting, reporting and persistent storage. Eight panels separate overview, patient records, appointments, queue, algorithms, reports, staff activity and resource calendars.

Scheduled and walk-in visits have persistent consultation records. Check-in joins the queue; Call next starts consultation; completion saves notes. Rescheduling checks conflicts, cancellation and no-show require reasons, and corrections retain previous notes. Clinician and room allocations reject overlapping active bookings. History and archiving preserve previous visits. A welcome screen opens recent workspaces or a fictional demonstration copy.

1.2 CLASS HIERARCHY AND RESPONSIBILITIES

PatientRecord is abstract, with pure virtual category(), consultationSummary() and clone() operations. GeneralPatient, EmergencyPatient and FollowUpPatient provide category-specific behaviour. Appointable defines scheduling and cancellation policies. Processing a PatientRecord reference dispatches to the concrete subtype at runtime, while clone() preserves that subtype in undo snapshots.

ClinicDatabase owns patient and appointment collections. ClinicManager applies validation, permissions and transactions. Separate panels present data; FileHandler and RecoveryService handle persistence. WorkspaceGuard detects competing file access. AccessControl and PortableBackup provide authentication and encryption. ReportService and Benchmark calculate reports and algorithm comparisons. Header/source files and namespaces separate these responsibilities.

1.3 DATA STRUCTURES

A std::vector of unique_ptr<PatientRecord> owns patients without object slicing. Appointments refer to immutable patient IDs and optional clinician/room names. A std::queue of QueueEntry objects preserves arrival order and links waiting patients to visits. A fixed std::array of ten CacheEntry objects tracks frequent access, replacing the least-used entry with an oldest-access tie-break.

A std::stack of UndoSnapshot objects stores up to fifty deep snapshots. Candidate transactions are validated before committing; rejected changes preserve live state and undo history. Undo restores the latest record state while retaining existing audit events and adding an Undo entry. Account changes and switching users clear the undo stack.

1.4 SEARCHING SORTING AND RECURSION

Linear search scans IDs and supports name/category filtering. Binary search uses an ID-sorted view prepared by the handwritten merge sort. Lookup costs O(log n); preparing the order costs O(n log n), reported separately in the interface.

Insertion sort has O(n) best-case and O(n squared) worst-case time. Recursive merge sort is stable with O(n log n) time and O(n) temporary storage. Ranges smaller than two items form its base case; recursive calls sort both halves before merging. Equal keys preserve their original order. Insertion sort suits small or nearly sorted lists. Merge sort suits larger lists and provides predictable performance. Required searching and sorting algorithms are implemented manually.

Both sorts support name, ID and age. The comparison runs sorted, reversed and seeded shuffled input, identical for each algorithm. One warm-up is excluded; five measured runs provide a median. Comparisons, moves and recursion depth supplement timing. A 2,000-record fictional dataset supports demonstrations. Background calculation keeps the interface responsive.

1.5 GUI DESIGN AND EVENTS

VBox and HBox arrange the page and action rows. Grid aligns booking controls, while Form pairs labels with fields. These layouts resize controls consistently. A QAbstractTableModel adapter presents patients. Signals and slots handle actions, selections and timers. Dynamic subtype fields, date/resource filters, duplicate warnings, validation messages, role-aware tooltips, printing and recovery dialogs support daily workflows. Text scaling, visible empty states and keyboard shortcuts improve usability. Search, sort, selection and scrolling survive updates.

1.6 VALIDATION SECURITY AND PERSISTENCE

Custom exceptions reject invalid contacts, IDs, ages, times, overlapping bookings, duplicate queue entries and malformed data. New appointments are future-dated; check-in requires the appointment date; completion requires an active consultation and notes. No-show requires a scheduled visit whose end time has passed. Follow-up selection uses a completed visit from the patient's history.

Administrator, clinician, receptionist and viewer roles govern actions. Clinical forms and tooltips restrict sensitive fields; report exports/printing require clinical or administrator access, and portable backups require an administrator. Staff passwords use salted PBKDF2-HMAC-SHA256. Windows DPAPI protects normal workspaces and recovery copies. Separate portable backups use a passphrase-derived key with AES-256-GCM and authenticated format metadata.

QSaveFile commits writes atomically. WorkspaceGuard reserves a canonical path with QLockFile and checks a SHA-256 fingerprint before saving. Autosave runs every thirty seconds, with unique files for unsaved sessions. Manual saves retain five protected previous versions. Recovery previews show timestamps and record counts; imported or recovered data opens unsaved for Save As.

1.7 CHALLENGES AND RESOLUTIONS

Polymorphic undo requires deep cloning of unique_ptr records. Views release stale pointers before transactions and restore selection by ID afterward. Walk-ins now create visit records so notes are retained. Resource validation checks patient, clinician and room conflicts together. An empty-state callback initially outlived its table viewport; parent ownership and guarded pointers corrected the teardown failure.

Portable encryption delegates AES to Windows CNG. Authentication, tamper rejection and an independent AES-GCM fixture verify the backup format. Normal DPAPI files remain bound to the Windows account; portable backups require their passphrase and staff administrator credentials. These local controls are an assessment implementation, not an independently evaluated clinical production system.

1.8 VERIFICATION

The final Qt Test run reports 31 results, 0 failures and 0 errors. Validation, workflow, permissions, persistence, recovery, algorithms and GUI behaviour are covered. Backup transfer to a second computer remains unverified.

1.9 PROJECT REPOSITORY

https://github.com/atanunu/COS-202-SCAPMS

The private repository contains src/ for C++ headers and implementation, data/ for fictional input files, tests/ for automated checks, scripts/ for build tools, output/screenshots/ for GUI evidence and docs/diagrams/ for UML source, PNG and PDF. Access must be granted to the assessor.